Activity against opinion: comparing the m-modes

In a polarized state, Fig. 3b of Baumann et al. (2020) shows how the agents' activity relates to their opinion: the more extreme an agent's opinion, the more active it tends to be, so the agents form a U shape. This notebook draws the same figure for each of the four m-modes of our NetLogo model (`netlogo/polarisation.nlogox`), to see whether the shape changes when the number of contacts depends on the opinion. It also counts how many runs ended polarized.

The data come from the BehaviorSpace experiment `activity-opinion` of the NetLogo model, saved as spreadsheet output. For every run it holds the parameters and two lists: the final opinions of all agents and their activities, both sorted by agent number.

Input and output:

- input: `data/activity-opinion/activity-opinion.csv`, the spreadsheet output of the experiment
- output: `results/activity-opinion/`, the figure and the table of polarized runs

```
input_file <- "../data/activity-opinion/activity-opinion.csv"
output_folder <- "../results/activity-opinion"
dir.create(output_folder, recursive = TRUE, showWarnings = FALSE)

modes <- c("uniform", "boost-extreme", "uprank-bridging", "uprank-bridging and downrank-toxic")
```

The data

The spreadsheet output starts with 6 lines about the experiment. After that, every row is one parameter or measure, and every run has three columns next to each other: the step at which the run ended, the opinions and the activities, in the order the reporters were entered in BehaviorSpace. The first column holds the row names, such as `m-mode` or `[run number]`. The parameters are written only in the first of a run's three columns, the other two are empty.

```
sheet <- read.csv(input_file, skip = 6, header = FALSE, colClasses = "character")
row_names <- sheet[, 1]
sheet <- sheet[, -1]
```

The row `[final value]` names what is in each column (`[step]`, then the two reporters), and the row below it holds the final values. So the columns named `[step]` are the first column of each run, and the opinions and activities are the next two columns:

```
final_names <- unlist(sheet[row_names == "[final value]", ])
final_values <- unlist(sheet[which(row_names == "[final value]") + 1, ])
first_column <- which(final_names == "[step]")

runs <- data.frame(
  mode = unlist(sheet[row_names == "m-mode", first_column]),
  run = as.integer(unlist(sheet[row_names == "[run number]", first_column])),
```

```
  opinions = final_values[first_column + 1],
  activities = final_values[first_column + 2]
)
```

NetLogo writes a list as a single piece of text, for example `[0.12 -1.5 3.4]`. `parse_list()` removes the brackets and splits the text at the spaces:

```
parse_list <- function(text) {
  without_brackets <- trimws(gsub("\\[|\\]", "", text))
  as.numeric(strsplit(without_brackets, "\\s+")[[1]])
}
```

We turn every run into a table with one row per agent, holding its opinion `x` and its activity `a`, and stack the runs on top of each other:

```
run_tables <- lapply(seq_len(nrow(runs)), function(k) {
  data.frame(
    mode = runs$mode[k],
    run = runs$run[k],
    x = parse_list(runs$opinions[k]),
    a = parse_list(runs$activities[k])
  )
})
agents <- do.call(rbind, run_tables)
```

BehaviorSpace numbers the runs across all modes. We number them 1, 2, ... within each mode instead:

```
agents$run <- ave(agents$run, agents$mode, FUN = function(run) match(run, sort(unique(run))))
```

Every mode should appear, with the same number of runs. The table below shows the number of agents per run for each mode, which should be N everywhere:

```
stopifnot(all(modes %in% agents$mode))
n_runs <- max(agents$run)
table(agents$mode) / n_runs
```

```
##
##                       boost-extreme                                uniform
##                                1000                                   1000
##                   uprank-bridging uprank-bridging and downrank-toxic
##                                1000                                   1000
```

The figure

For each mode we draw a 2-D histogram: the opinion axis is cut into bins of width 0.2, the activity axis (from 0 to 1) into 40 bins, and every cell is coloured by how many agents fall into it. The count is divided by the number of runs, so it is the number of agents per cell in one run. The colours use a log scale because the counts range over several orders of magnitude. Empty cells are left white.

All panels use the same bins and the same colour scale, so their colours can be compared directly.

```r
x_bin_width <- 0.2
x_max <- ceiling(max(abs(agents$x)) / x_bin_width) * x_bin_width
x_breaks <- seq(-x_max, x_max, by = x_bin_width) # symmetric around 0
a_breaks <- seq(0, 1, length.out = 41)
```

```r
# log10 of the number of agents per cell in one run; NA for empty cells
log_density <- function(mode) {
  in_mode <- agents[agents$mode == mode, ]
  counts <- table(
    cut(in_mode$x, x_breaks, include.lowest = TRUE),
    cut(in_mode$a, a_breaks, include.lowest = TRUE)
  )
  per_run <- unclass(counts) / n_runs
  per_run[per_run == 0] <- NA
  log10(per_run)
}

densities <- lapply(modes, log_density)
colour_range <- range(unlist(densities), na.rm = TRUE)
colours <- hcl.colors(100, "Spectral")
```

The opinion axis runs from -6 to 6 in every panel except boost-extreme. That mode produces far more extreme opinions, so its panel shows the whole range of its own opinions. On an axis that wide the other panels would shrink to a thin strip in the middle. In the other modes, agents with $|x| > 6$ (if there are any) fall outside the plot.

The four panels, labelled a to d, sit in a 2 x 2 grid, with the colour bar below them.

```r
figure_file <- file.path(output_folder, "activity-opinion.png")

png(figure_file, width = 2000, height = 1800, res = 200)
layout(matrix(c(1, 2, 3, 4, 5, 5), nrow = 3, byrow = TRUE), heights = c(1, 1, 0.25))
par(mar = c(4, 4, 3, 1))

for (k in seq_along(modes)) {
  if (modes[k] == "boost-extreme") {
    x_limit <- max(abs(agents$x[agents$mode == modes[k]]))
  } else {
    x_limit <- 6
  }
  image(x_breaks, a_breaks, densities[[k]],
    col = colours, zlim = colour_range, xlim = c(-x_limit, x_limit),
    xlab = "opinion x", ylab = "activity a", main = modes[k]
  )
  mtext(letters[k], side = 3, line = 1, adj = 0, font = 2, cex = 1.2)
}

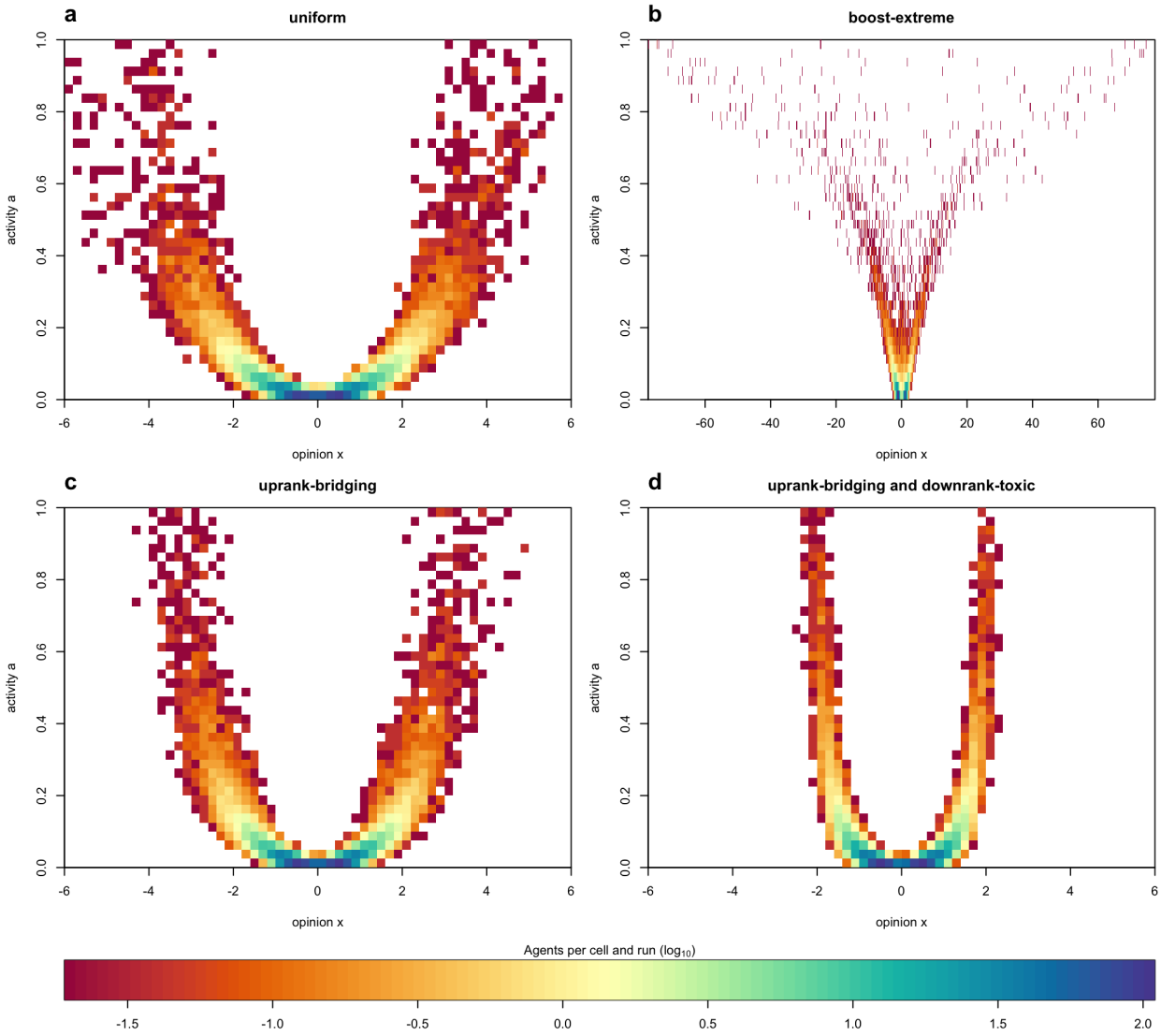
# colour bar
par(mar = c(3, 4, 2, 1))
bar_values <- seq(colour_range[1], colour_range[2], length.out = 100)
image(bar_values, 1, matrix(bar_values, ncol = 1),
  col = colours, yaxt = "n", xlab = "", ylab = ""
)
```

```
title(expression("Agents per cell and run (log"[10] * ")"), line = 0.6, font.main = 1, cex.main = 1)

dev.off()
```

```
## pdf
##   2
```

```
knitr::include_graphics(figure_file)
```



Polarized runs

The figure pools all runs of a mode, polarized or not. To know how many runs actually ended polarized, we count a run as polarized if at least 20% of the agents end up on each side, that is, if the share of agents with a positive opinion lies between 0.2 and 0.8. This is our own rule; the paper doesn't say how it picked its polarized states.

For each run we compute that share and the mean absolute opinion $|x|$, which tells how extreme the opinions are on average:

```
per_run <- aggregate(
  cbind(share_positive = x > 0, mean_abs_x = abs(x)) ~ mode + run,
  data = agents, FUN = mean
)
per_run$polarized <- per_run$share_positive >= 0.2 & per_run$share_positive <= 0.8
```

Per mode: the number of polarized runs out of all runs, and the mean and standard deviation of the mean $|x|$ over the runs. The table is also saved as `polarized-runs.csv`.

```
summary_table <- data.frame(
  mode = modes,
  n_polarized = sapply(modes, function(mode) sum(per_run$polarized[per_run$mode == mode])),
  n_runs = sapply(modes, function(mode) sum(per_run$mode == mode)),
  mean_abs_x = sapply(modes, function(mode) mean(per_run$mean_abs_x[per_run$mode == mode])),
  sd_abs_x = sapply(modes, function(mode) sd(per_run$mean_abs_x[per_run$mode == mode]))
)
write.csv(summary_table, file.path(output_folder, "polarized-runs.csv"), row.names = FALSE)

knitr::kable(summary_table, row.names = FALSE, digits = 3)
```

mode	n_polarized	n_runs	mean_abs_x	sd_abs_x
uniform	50	50	0.786	0.036
boost-extreme	1	50	1.640	0.346
uprank-bridging	50	50	0.749	0.036
uprank-bridging and downrank-toxic	50	50	0.682	0.027